

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 4
DESIGN PATTERN & OBJECT COMPOSING**

**DISUSUN OLEH :
BAGUS DWI PUTRA SETIAWAN - 2350081042**



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR	ii
BAB I. HASIL PRAKTIKUM.....	1
I.1 Latihan MVC Student	1
I.1.A Langkah Kerja / Source Code	1
I.1.B Screenshoot	4
I.1.C Analisis.....	4
BAB II. TUGAS PRAKTIKUM	6
II.1 Tugas 1 MVC Course.....	6
II.1.A Langkah Kerja / Source Code	6
II.1.B Screenshoot	11
II.1.C Analisis.....	11
II.2 Tugas 2 MVC Kasir	11
II.2.A Langkah Kerja / Source Code	11
II.2.B Screenshoot	16
II.2.C Analisis.....	16
BAB III. KESIMPULAN.....	17

DAFTAR GAMBAR

Gambar 1. 1 Hasil Latihan MVC Student.....	4
Gambar 2. 1 Hasil Tugas MVC Course	11
Gambar 2. 2 Hasil Tugas MVC Kasir.....	16

BAB I. HASIL PRAKTIKUM

I.1 Latihan MVC Student

I.1.A Langkah Kerja / Source Code

Class Student.java

```
public class Student {  
    private String rollNo;  
    private String name;  
  
    public String getRollNo() {  
        return rollNo;  
    }  
  
    public void setRollNo(String rollNo) {  
        this.rollNo = rollNo;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Class StudentView.java

```
public class StudentView {
```

```
        public void printStudentDetails(String
studentName, String studentRollNo) {
            System.out.println("Student: ");
            System.out.println("Name: " +
studentName);
            System.out.println("Roll No: " +
studentRollNo);
        }
    }
}
```

Class StudentController.java

```
public class StudentController {
    private Student model;
    private StudentView view;

    public StudentController(Student model,
StudentView view) {
        this.model = model;
        this.view = view;
    }

    public void setStudentName(String name) {
        model.setName(name);
    }

    public String getStudentName() {
        return model.getName();
    }
}
```

```
        public void setStudentRollNo(String rollNo) {
            model.setRollNo(rollNo);
        }

        public String getStudentRollNo() {
            return model.getRollNo();
        }

        public void updateView() {
            view.printStudentDetails(model.getName(),
model.getRollNo());
        }
    }
}
```

Class MVCPatternDemo.java

```
public class MVCPatternDemo {
    public static void main(String[] args) {
        Student model =
retrieveStudentFromDatabase();
        StudentView view = new StudentView();
        StudentController controller = new
StudentController(model, view);

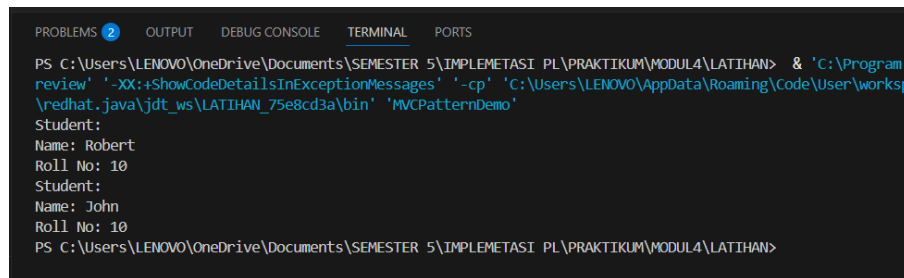
        controller.updateView();
        controller.setStudentName("John");
        controller.updateView();
    }
}
```

```

        private static Student
retrieveStudentFromDatabase() {
            Student student = new Student();
            student.setName("Robert");
            student.setRollNo("10");
            return student;
        }
    }
}

```

I.1.B Screenshoot



```

PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMETASI PL\PRAKTIKUM\MODUL4\LATIHAN> & 'C:\Program Files\Java\jdk-11.0.10\bin\java.exe' -XX:ShowCodeDetailsInExceptionMessages' -cp' 'C:\Users\LENOVO\AppData\Roaming\Code\User\workspace\redhat.java\jdt_ws\LATIHAN_75e8cd3a\bin' 'MVCPatternDemo'
Student:
Name: Robert
Roll No: 10
Student:
Name: John
Roll No: 10
PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMETASI PL\PRAKTIKUM\MODUL4\LATIHAN>

```

Gambar 1. 1 Hasil Latihan MVC Student

I.1.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu menerapkan konsep Model-View-Controller (MVC) untuk memisahkan antara logika data, tampilan, dan kontrol alur program. Kelas Student berfungsi sebagai Model yang menyimpan data mahasiswa seperti nama dan nomor induk, sedangkan StudentView bertugas untuk menampilkan informasi tersebut ke konsol tanpa terlibat langsung dalam proses pengolahan data. Hal ini menjadikan setiap komponen memiliki peran yang jelas dan spesifik.

Kelas StudentController bertindak sebagai penghubung yang menjembatani komunikasi antara model dan view. Controller memastikan data dari model

dapat ditampilkan dengan benar di view, serta memungkinkan perubahan data dilakukan dengan aman tanpa merusak struktur program.

BAB II. TUGAS PRAKTIKUM

II.1 Tugas 1 MVC Course

Melengkapi Code dan Jalankan di IDE

II.1.A Langkah Kerja / Source Code

Output :

```
Course Details:
Name: Java
Course ID: 01
Course Category: Programming

Tampilan Setelah diupdate
Course Details:
Name: Python
Course ID: 01
Course Category: Programming
```

Class MVCPatternDemo.java

```
public class MVCPatternDemo {
    public static void main(String[] args) {
        // fetch student record based on his roll
no from the database
        Course model =
retrieveCourseFromDatabase();

        // Create a view : to write course details
on console
        CourseView view = new CourseView();
```

```

        CourseController controller = new
CourseController(model, view);

        controller.updateView();

        // update model data
        controller.setCourseName("Python");
        System.out.println("\nAfter updating,
Course Details are as follows");
        controller.updateView();
    }

    private static Course
retrieveCourseFromDatabase() {
        Course course = new Course();
        course.setName("Java");
        course.setId("01");
        course.setCategory("Programming");
        return course;
    }
}

```

Class Course.java

```

public class Course {
    private String CourseName;
    private String CourseId;
    private String CourseCategory;

    public String getId() {

```

```
        return CourseId;
    }

    public void setId(String id) {
        CourseId = id;
    }

    public String getName() {
        return CourseName;
    }

    public void setName(String name) {
        CourseName = name;
    }

    public String getCategory() {
        return CourseCategory;
    }

    public void setCategory(String category) {
        CourseCategory = category;
    }
}
```

Class CourseView.java

```
public class CourseView {
    public void printCourseDetails(String
CourseName, String CourseId, String
CourseCategory) {
```

```
        System.out.println("Course Details: ");
        System.out.println("Name: " + CourseName);
        System.out.println("Course ID: " +
CourseId);
        System.out.println("Course Category: " +
CourseCategory);
    }
}
```

Class CourseController.java

```
public class CourseController {
    private Course model;
    private CourseView view;

    public CourseController(Course model,
CourseView view) {
        this.model = model;
        this.view = view;
    }

    public void setCourseName(String name) {
        model.setName(name);
    }

    public String getCourseName() {
        return model.getName();
    }

    public void setCourseId(String id) {
```

```
        model.setId(id);
    }

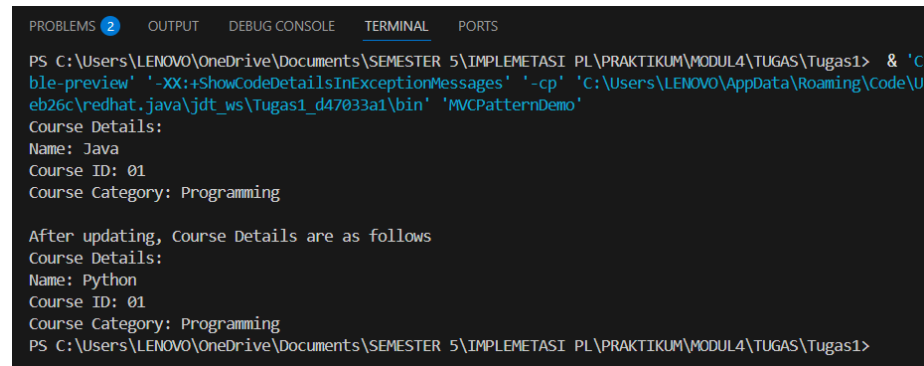
    public String getCourseId() {
        return model.getId();
    }

    public void setCourseCategory(String category)
    {
        model.setCategory(category);
    }

    public String getCourseCategory() {
        return model.getCategory();
    }

    public void updateView() {
        view.printCourseDetails(model.getName(),
model.getId(), model.getCategory());
    }
}
```

II.1.B Screenshoot



```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMETASI PL\PRAKTIKUM\MODUL4\TUGAS\Tugas1> & 'C:\Program Files\Java\jdk-11.0.10\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\LENOVO\AppData\Roaming\Code\U
eb26c\redhat.java\jdt_ws\Tugas1_d47033a1\bin' 'MVCPatternDemo'
Course Details:
Name: Java
Course ID: 01
Course Category: Programming

After updating, Course Details are as follows
Course Details:
Name: Python
Course ID: 01
Course Category: Programming
PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMETASI PL\PRAKTIKUM\MODUL4\TUGAS\Tugas1>
```

Gambar 2. 1 Hasil Tugas MVC Course

II.1.C Analisis

Hasil analisis yang saya dapat pada Tugas ini yaitu melengkapi code yang belum diisi dari tugas yang diberikan. Kelas Course berperan sebagai Model yang menyimpan informasi penting seperti nama kursus, ID, dan kategori. Kelas CourseView berfungsi menampilkan data tersebut dengan format terstruktur ke dalam konsol, sementara seluruh logika pembaruan data dikelola oleh CourseController.

Pemisahan ini menghindari tumpang tindih fungsi antar komponen program. Controller bertugas memperbarui data model dan memanggil metode view agar data terbaru dapat ditampilkan dengan benar.

II.2 Tugas 2 MVC Kasir

Mengubah Code Menjadi Pattern MVC

II.2.A Langkah Kerja / Source Code

Class KasirModel.java

```
public class KasirModel {
    private String namaBarang;
    private int hargaBarang;
```

```
private int stokBarang;

public String getNamaBarang() {
    return namaBarang;
}

public void setNamaBarang(String namaBarang) {
    this.namaBarang = namaBarang;
}

public int getHargaBarang() {
    return hargaBarang;
}

public void setHargaBarang(int hargaBarang) {
    this.hargaBarang = hargaBarang;
}

public int getStokBarang() {
    return stokBarang;
}

public void setStokBarang(int stokBarang) {
    this.stokBarang = stokBarang;
}

public void kurangiStok(int jumlah) {
    this.stokBarang -= jumlah;
}
```

```
        public int hitungTotalBayar(int jumlah) {  
            return hargaBarang * jumlah;  
        }  
    }  
}
```

Class KasirView.java

```
import java.util.Scanner;  
  
public class KasirView {  
    private Scanner input = new  
Scanner(System.in);  
  
    public void tampilkanInfoBarang(String nama,  
int harga, int stok) {  
        System.out.println("Nama Barang : " +  
nama);  
        System.out.println("Harga Barang : " +  
harga);  
        System.out.println("Stok Barang : " +  
stok);  
    }  
  
    public int inputJumlahPembelian() {  
        System.out.println("Transaksi Pembelian");  
        System.out.print("Jumlah barang : ");  
        return input.nextInt();  
    }  
  
    public void tampilkanTotalBayar(int total) {
```



```
        System.out.println("Jumlah Bayar : " +
total);
    }
}
```

Class KasirController.java

```
public class KasirController {
    private KasirModel model;
    private KasirView view;

    public KasirController(KasirModel model,
KasirView view) {
        this.model = model;
        this.view = view;
    }

    public void tampilkanBarang() {

view.tampilkanInfoBarang(model.getNamaBarang(),
model.getHargaBarang(), model.getStokBarang());
    }

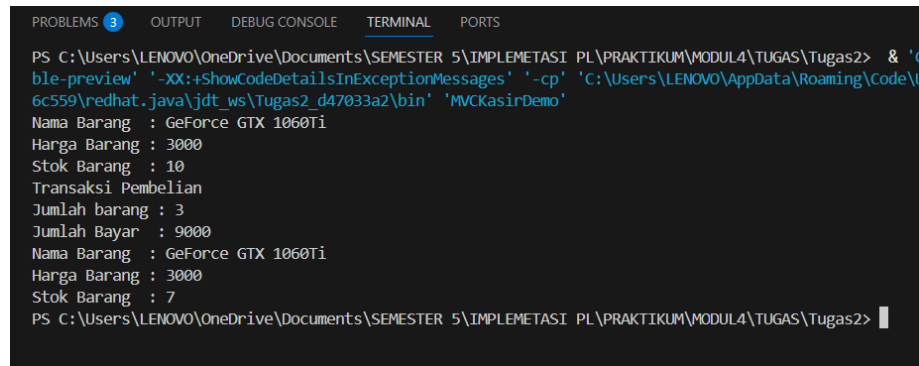
    public void prosesTransaksi() {
        int jumlahBeli =
view.inputJumlahPembelian();
        model.kurangiStok(jumlahBeli);
        int total =
model.hitungTotalBayar(jumlahBeli);
        view.tampilkanTotalBayar(total);
    }
}
```

```
}  
}
```

Class MVCKasirDemo.java

```
public class MVCKasirDemo {  
    public static void main(String[] args) {  
        // Model  
        KasirModel model = new KasirModel();  
        model.setNamaBarang("GeForce GTX 1060Ti");  
        model.setHargaBarang(3000);  
        model.setStokBarang(10);  
  
        // View  
        KasirView view = new KasirView();  
  
        // Controller  
        KasirController controller = new  
KasirController(model, view);  
  
        // Proses  
        controller.tampilkanBarang();  
        controller.prosesTransaksi();  
        controller.tampilkanBarang();  
    }  
}
```

II.2.B Screenshoot



```
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMETASI PL\PRAKTIKUM\MODUL4\TUGAS\Tugas2> & 'C:\Program Files\Java\jdk-11.0.10\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\LENOVO\AppData\Roaming\Code\User\globalStorage\redhat.java\jdt_ws\Tugas2_d47033a2\bin' 'MVCKasirDemo'
Nama Barang : GeForce GTX 1060Ti
Harga Barang : 3000
Stok Barang : 10
Transaksi Pembelian
Jumlah barang : 3
Jumlah Bayar : 9000
Nama Barang : GeForce GTX 1060Ti
Harga Barang : 3000
Stok Barang : 7
PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMETASI PL\PRAKTIKUM\MODUL4\TUGAS\Tugas2>
```

Gambar 2. 2 Hasil Tugas MVC Kasir

II.2.C Analisis

Hasil analisis yang saya dapat pada Tugas ini yaitu mengubah program kasir dari struktur prosedural menjadi struktur berbasis MVC agar lebih rapi dan terorganisir. KasirModel menyimpan seluruh data barang, stok, dan logika perhitungan seperti pengurangan stok serta total harga. KasirView bertugas menampilkan informasi barang, menerima input jumlah pembelian, serta menampilkan total pembayaran yang harus dibayar pelanggan.

KasirController mengatur alur kerja program, mulai dari menampilkan data, menerima input, menghitung transaksi, hingga memperbarui tampilan akhir. Penerapan MVC membuat kode menjadi lebih modular, mudah diuji, dan terhindar dari duplikasi logika.

BAB III. KESIMPULAN

Kesimpulan yang dapat saya ambil dalam pertemuan keempat pada Praktikum Implementasi Perangkat Lunak adalah kita dapat memahami dan mempraktikkan konsep Design Pattern serta Object Composing dalam pengembangan perangkat lunak berbasis objek. Design Pattern membantu pengembang menyelesaikan masalah umum dalam desain program dengan solusi yang efisien dan teruji, seperti pola Interface Segregation Principle (ISP) dan Dependency Inversion Principle (DIP). Melalui latihan dan tugas pada modul ini, dipelajari bagaimana menerapkan prinsip-prinsip tersebut dalam kode agar lebih fleksibel, mudah diperluas, serta meminimalkan ketergantungan antar kelas. Penerapan Object Composing juga memperkuat pemahaman tentang hubungan antar objek dalam membangun sistem yang modular dan terstruktur. Konsep MVC (Model-View-Controller), yaitu pola arsitektur yang memisahkan logika bisnis (Model), tampilan antarmuka (View), dan pengendali alur data (Controller).